# Guia para implementação do controle de estabilização de voo de um quadricóptero utilizando Arduino

Angélico, R.A., Daud Filho, A.C., Bidinotto, J.H.

Julho / 2025

**Resumo**

Essas notas visam apresentar os principais tópicos referentes a implementação de um controle PID para estabilização de um quadricóptero. Os desenvolvimentos aqui descritos são referentes a quatro etapas de desenvolvimento. A última etapa corresponde a implementação do controle de estabilização.

# Sumário

**1 Variáveis globais** **3**

**2 Etapa I: Determinação da atitude da aeronave** **3**

2.1 Redução de ruído do sinal através de um filtro digital . . . . . 3

2.2 Determinação dos ângulos de Euler a partir de integração das velocidades angulares dos giroscópios . . . . . 3

2.3 Determinação dos ângulos de Euler a partir dos acelerômetros . . . . . 4

2.4 Filtro complementar . . . . . 4

**3 Etapa II: Medição das larguras de pulso dos sinais** **6**

**4 Etapa III: Teste dos motores** **7**

4.1 Configuração de partida / parada . . . . . 8

**5 Etapa IV: Estabilização da Aeronave pelo Controle PID** **9**

5.1 Implementação do controlador PID . . . . . 9

5.2 Leitura da voltagem da bateria . . . . . 15

5.3 Geração dos Pulsos aos ESCs . . . . . 16

5.4 Visão geral do código . . . . . 17

**A Códigos auxiliares** **19**

A.1 Configuração da IMU . . . . . 19

A.2 Leitura dos valores da IMU . . . . . 20

A.3 Interrupções para leitura dos códigos . . . . . 20

**B Variáveis globais** **22**

# 1 Variáveis globais

As variáveis apresentados no Apêndice B referem-se as variáveis globais utilizadas no desenvolvimento do código. Evite alterar o nome das variáveis.

# 2 Etapa I: Determinação da atitude da aeronave

Esse código visa a identificação da atitude a partir dos dados dos acelerômetros e giroscópios da IMU (*Inertial Measurement Unit)*. Os ângulos de Euler serão obtidos a partir da combinação dos dados oriundos desses sensores. A seguir, apresentam-se as principais equações referentes ao tratamento dos dados da IMU.

O procedimento para estabelecimento dos fundos de escala e filtros da IMU estão disponíveis no Apêndice A.1. O acesso dos dados brutos da IMU podem ser obtidos utilizando o Apêndice A.2.

## 2.1 Redução de ruído do sinal através de um filtro digital

Os valores brutos vindos dos giroscópios possuem bastante ruído, assim utilizamos um filtro digital para cada eixo para atenuar esse efeito. As equações relativas ao filtro são mostradas a seguir, cujo valor final é a soma de dois termos, sendo que a soma dos coeficientes deve ser igual a 1, i.e.

$$p^{k+1} = 0.7\, p^k + 0.3\, p_{new} \tag{1}$$

$$q^{k+1} = 0.7\, q^k + 0.3\, q_{new} \tag{2}$$

$$r^{k+1} = 0.7\, r^k + 0.3\, r_{new} \tag{3}$$

onde $p$, $q$ e $r$ denotam as velocidades angulares em torno dos eixos $x$, $y$ e $z$ do corpo, obtidas através dos giroscópios da IMU. As variáveis com o subíndice *new* expressam os novos valores obtidos dos sensores. Aquelas com sobrescrito $k$, referem-se aos valores do passo anterior, e por fim, as com sobrescrito $k+1$, os valores do passo de tempo atual.

## 2.2 Determinação dos ângulos de Euler a partir de integração das velocidades angulares dos giroscópios

A taxa de variação dos ângulos de Euler estão relacionadas às velocidades angulares:

$$\begin{bmatrix} \dot{\phi}^{k+1}_{gyro} \\ \dot{\theta}^{k+1}_{gyro} \\ \dot{\psi}^{k+1}_{gyro} \end{bmatrix} = \begin{bmatrix} 1 & \tan\theta^k \sin\phi^k & \tan\theta^k \cos\phi^k \\ 0 & \cos\phi^k & -\sin\phi^k \\ 0 & \sin\phi^k \sec\theta^k & \cos\phi^k \sec\theta^k \end{bmatrix} \begin{bmatrix} p^{k+1} \\ q^{k+1} \\ r^{k+1} \end{bmatrix} \tag{4}$$

onde $\phi$ é o ângulo de rolagem, $\theta$ é o ângulo de arfagem e $\psi$ o ângulo de guinada.

A partir da integração numérica da Eq.(4) em cada um dos ângulos, obtêm-se:

$$\phi^{k+1} = \phi^k + \dot{\phi}^{k+1}_{gyro}\,\Delta t = \phi^k + (p^{k+1} + q^{k+1}\tan\theta^k \sin\phi^k + r^{k+1}\tan\theta^k \cos\phi^k)\,\Delta t \tag{5}$$

$$\theta^{k+1} = \theta^k + \dot{\theta}^{k+1}_{gyro}\,\Delta t = \theta^k + (q^{k+1}\cos\phi^k - r^{k+1}\sin\phi^k)\,\Delta t \tag{6}$$

$$\psi^{k+1} = \psi^k + \dot{\psi}^{k+1}_{gyro}\,\Delta t = \psi^k + (q^{k+1}\sin\phi^k\,\sec\theta^k + r^{k+1}\cos\phi^k\,\sec\theta^k)\,\Delta t \tag{7}$$

É importante ressaltar aqui que $\phi^k$, $\theta^k$ e $\psi^k$ são valores de ângulos de Euler resultado da integração dos passos anteriores dos dados advindos apenas dos giroscópios, mas será posteriormente mostrado com a implementação do filtro complementar que esses termos também serão advindos dos acelerômetros.

### 2.3 Determinação dos ângulos de Euler a partir dos acelerômetros

A IMU possui três acelerômetros, alinhados com os eixos coordenados xyz, de forma que na condição em que a IMU permanecer em uma superfície plana estacionária alinhada com a Terra e eixo z ortogonal, a leitura dos acelerômetros deverá ser ($Acc_x$, $Acc_y$, $Acc_z$) = $(0, 0, g)$, sendo $g$ o valor da aceleração da gravidade. Portanto, a medida que a IMU mudar sua orientação espacial, as medidas dos acelerômetros nos eixos xyz serão uma projeção da aceleração da gravidade nesses eixos.

Deste modo, podemos calcular os ângulos de Euler através das equações a seguir. Primeiramente, o valor da aceleração total medida, i.e.:

$$Acc_{Total} = \sqrt{Acc_x^2 + Acc_y^2 + Acc_z^2} \tag{8}$$

O ângulo de rolagem $\phi$, calculado de forma aproximada pela projeção da medida da aceleração no eixo y em relação ao vetor aceleração total:

$$\phi^{k+1}_{acc} = \arcsin\left(\frac{Acc_y}{Acc_{Total}}\right) \tag{9}$$

O ângulo de arfagem $\theta$, calculado de forma aproximada pela projeção da medida da aceleração no eixo x em relação ao vetor aceleração total, i.e.:

$$\theta^{k+1}_{acc} = -\arcsin\left(\frac{Acc_x}{Acc_{Total}}\right) \tag{10}$$

### 2.4 Filtro complementar

A atitude da aeronave é descrita através dos ângulos de Euler, e estes podem ser medidos de duas formas utilizando os dados advindos da IMU, ou seja, podem ser medidos através da integração das

velocidades angulares, e pela decomposição das acelerações medidas nos eixos xyz em relação ao vetor aceleração total, que para a situação estacionária equivaleria à aceleração da gravidade.

Contudo, ambas formas possuem vantagens e desvantagens. Ao se realizar a integração numérica das velocidades ocorrem necessariamente erros numéricos que vão se somando, o que pode ser identificado ao se mover o quadricóptero, e em seguida retornar à orientação inicial. Será demonstrado que os ângulos de Euler finais não serão equivalentes aos iniciais. Assim, os dados dos giroscópios são mais confiáveis em movimentos de alta frequência.

Além disso, os acelerômetros da IMU medem todas as acelerações atuantes na aeronave, ou seja, não medem com exclusividade a aceleração da gravidade. Desta forma, movimentos que ocorrem com altas acelerações podem prejudicar o cálculo da decomposição das acelerações medidas em relação ao vetor aceleração da gravidade. Portanto, a utilização das medidas das acelerações são mais apropriadas em movimentos de baixa frequência, ou seja, de longo prazo.

Assim sendo, o filtro complementar busca combinar o melhor das duas formas de calcular os ângulos de Euler. As equações finais são mostradas a seguir para o ângulo de rolagem e de arfagem:

$$\phi^{k+1} = 0.9996\,(\phi^{k} + \dot{\phi}^{k+1}_{gyro}\Delta t) + 0.0004\,\phi^{k+1}_{acc} \tag{11}$$

$$\theta^{k+1} = 0.9996\,(\theta^{k} + \dot{\theta}^{k+1}_{gyro}\Delta t) + 0.0004\,\theta^{k+1}_{acc} \tag{12}$$

Observe que os ângulos finais são devidos à combinação de dois termos cuja soma dos coeficientes deve ser igual a 1. Os coeficientes usados aqui foram 0.9996 e 0.0004, mas esses coeficientes podem ser outros. O primeiro termo, que possui coeficiente mais alto é o termo de mais alta frequência e carrega os valores de ângulos anteriores, enquanto o segundo termo, de menor coeficiente é o termo de baixa frequência.

Informações relevantes:

- As equações acima são descritas para um sistema de coordenadas cujo eixo $x$ está orientado do CG ao nariz da aeronave, eixo $y$ do CG para o lado direito da aeronave (asa direita), e eixo $z$ do CG para o centro da Terra. Os valores obtidos a partir da IMU devem estar de acordo com esse sistema de coordenadas, ou seja, ângulo de rolagem ($\phi$) positivo para lado esquerdo para cima, ângulo de arfagem ($\theta$) positivo para nariz para cima e ângulo de guinada ($\psi$) positivo para sentido horário quando a aeronave vista de cima.
- Os valores das velocidade angulares medidas pelos giroscópios expressas em graus por segundo.
- Os valores das velocidades angulares já devem estar corrigidos em relação aos erros de *offset*
- É possível que ao se instalar a IMU na aeronave a orientação dos eixos dos acelerômetros e giroscópios embarcados não estejam nos sentidos corretos com relação aos eixos coordenados de referência xyz fixos no corpo. Assim, caso isto tenha ocorrido, os ângulos de Euler calculados podem resultar com sinais invertidos. Para corrigir esse problema, multiplique por -1 os sinais dos acelerômetros e giroscópios, ou ainda dos ângulos calculados, que estejam invertidos.

- A fim de verificar se código está coerente, utilize o recurso *plotter serial* ou *monitor serial* do Arduino para visualizar os ângulos de Euler resultantes.

## 3 Etapa II: Medição das larguras de pulso dos sinais

Esse código visa a identificação da largura de pulso de cada um dos canais do rádio, a implementação de uma rotina de normalização do sinal, bem como o armazenamento dos valores extremos e centrais na EEPROM.

Os valores a serem escritos e lidos da EEPROM são inteiros (2 bytes). Os códigos abaixo apresentam um exemplo de como escrever o número inteiro $N$ nas posições $ads$ e $ads + 1$ da EEPROM.

**Exemplo de escrita de dados na EEPROM**

```
1  EEPROM.write(ads, N & 0b11111111);
2  EEPROM.write(ads+1, N >> 8);
```

**Exemplo de leitura dos dados da EEPROM**

```
1  N = ((EEPROM.read(ads+1) << 8) | EEPROM.read(ads))
```

Na EEPROM serão salvos os dados referentes as posições central, e os limites inferiores e superiores de cada um dos canais. A Tabela 1 apresenta os valores e os locais que esses serão armazenados na EEPROM.

Tabela 1: Endereços da EEPROM para armazenamento das posições central, inferior e superior dos canais.

| Canais | Centro | Inferior | Superior |
|---|---|---|---|
| THRO | 0, 1 | 8, 9 | 16, 17 |
| AILE | 2, 3 | 10, 11 | 18, 19 |
| ELEV | 4, 5 | 12, 13 | 20, 21 |
| RUDD | 6, 7 | 14, 15 | 22, 23 |

A leitura da largura dos sinais provenientes do receptor do rádio será realizada utilizando-se interrupções. A configuração das interrupções deve ser realizada na função *setup* do código do Arduino e a leitura da largura de pulso de cada um dos canais a partir da função ISR. O Apêndice A.3 apresenta a função ISR para leitura da largura de pulso do receptor, considerando que esse está conectado nas portas D8 (THRO - Tração), D9 (AILE - Rolagem), D10 (ELEV - Arfagem) e D11 (RUDD - Guinada) do Arduino.

A configuração das interrupções na função *setup* é realizada a partir das linhas de código a seguir:

```
PCICR |= (1 << PCIE0); // set PCIE0 to enable PCMSK0 scan.
PCMSK0 |= (1 << PCINT0); // set PCINT0 to trigger an interrupt on state change.
PCMSK0 |= (1 << PCINT1); // set PCINT1 to trigger an interrupt on state change.
PCMSK0 |= (1 << PCINT2); // set PCINT2 to trigger an interrupt on state change.
PCMSK0 |= (1 << PCINT3); // set PCINT3 to trigger an interrupt on state change.
```

A identificação do limite inferior, do limite superior e do centro deve ser realizada a partir da interação com o usuário. Ao ser executado, o código deve pedir ao usuário que movimente os comandos seguindo determinadas instruções, exibindo, ao final, os valores identificados. O procedimento de identificação dos limites deve estar implementado na função *setup*.

Posteriormente, na função *loop*, deve-se ler os sinais dos diferentes canais, normalizá-los e exibir os valores a partir da saída serial para leitura na tela.

Nominalmente, os sinais recebidos pelos canais do rádio tem uma largura mínima de $1000\mu s$ e máxima de $2000\mu s$, na frequência de 45 Hz. Entretanto, os sinais recebidos diferem desses valores previamente estabelecidos, sendo assim, necessário realizar um procedimento de normalização dos sinais. Seja *signal* o valor recebido, em largura de pulso em $\mu s$, pela função de interrupção correspondente a um dos canais. A normalização de *signal*, *signal_norm*, para valores de $1000\mu s$ - 2000 $\mu s$ deve seguir um modelo linear entre o centro e os limites, i.e.

$$signal_norm = \begin{cases} 1500 - 500\,\dfrac{(center - signal)}{(center - low)}, & signal < center \\ 1500 + 500\,\dfrac{(signal - center)}{(high - center)}, & signal > center \\ 1500, & else \end{cases} \tag{13}$$

onde *low*, *center* e *high* corresponde as larguras de pulso dos controles nas posições inferior, central e superior, respectivamente.

## 4 Etapa III: Teste dos motores

Essa etapa visa o teste dos motores individualmente e a verificação dos dados armazenados na EEPROM. A verificação dos dados armazenados na EEPROM deve ser realizada na função *setup*, enquanto o teste dos motores na função *loop*.

No processo de verificação dos dados armazenados na EEPROM, deve-se ler os dados armazenados na EEPROM e imprimi-los para visualização do usuário. No teste dos motores, o usuário poderá digitar o valor do motor a ser testado, o qual pode variar de 1 a 5. Os números de 1 a 4 correspondem aos motores FR, RR, RL, FL, respectivamente. O número 5 corresponde ao teste de todos os motores simultaneamente. A largura de pulso deve ser controlada pelo canal referente ao (THRO - Tração).

## 4.1 Configuração de partida / parada

A configuração de partida / parada dos motores do quadricóptero é realizada a partir de uma sequência de comandos de rádio que, consequentemente, modificam uma variável (*start*) que estabelece as faixas de largura de sinais permitidas aos motores. Aqui, será utilizada uma combinação dos sinais de rádio (THRO - Tração) e (RUDD - Guinada), ou seja, apenas os canais do *stick* esquerdo do transmissor de rádio.

A inicialização dos motores ocorrerá quando ambos os sinais estiverem com a menor largura de pulso e então (RUDD - Guinada) retornar ao centro, e a parada quando a largura de pulso do (THRO - Tração) for a menor e a largura do (RUDD - Guinada) a máxima. Isto é, o movimento do *stick* esquerdo do transmissor de rádio de mínima Tração para a esquerda e depois retornar ao centro deve iniciar os motores, e então o movimento de mínima Tração para a direita deve desligar os motores.

- Assim, a variável *start* inicia-se no valor $start = 0$, onde os motores devem permanecer desligados, ou seja, largura de pulso aos quatro ESCs de 1000 $\mu s$.
- Ao se emitir a combinação de comando de rádio de (THRO - Tração) mínima e (RUDD - Guinada) mínima a variável passa a ser $start = 1$, mantendo os motores desligados (isso evita que os motores se iniciem com comando de guinada máximo).
- Após o retorno do (RUDD - Guinada) à posição central, mantendo (THRO - Tração) mínima, o valor da variável é alterado para $start = 2$, onde os motores devem ser iniciados, ou seja, o valor mínimo de largura de pulso aos ESCs passa a ser 1100 $\mu s$, o que irá iniciar o giro dos motores na velocidade mais baixa. Além disso, ao se iniciar os motores é importante também reseter (zerar) os contadores de erros dos controladores PID, ou seja, o somatório do erro de integrador e o último erro para o cálculo do erro derivativo, bem como determinar que a atitude iniciada é equivalente aos ângulos devido aos acelerômetros.
- Caso o (RUDD - Guinada) esteja com a máxima largura de pulso e o (THRO - Tração) com largura mínima, a variável deve ser alterada novamente para $start = 0$, o que irá desligar os motores.

A configuração é realizada a partir da inserção das seguintes linhas de código:

```
if(receiver_input_channel_1 < 1050 && receiver_input_channel_4 < 1050)
   start = 1;

if(start == 1 && receiver_input_channel_1 < 1050 && receiver_input_channel_4 > 1450) {
          start = 2;
  // Set the IMU pitch angle equal to the accelerometer pitch angle when the quadcopter is started.
  angle_pitch = angle_pitch_acc;
  // Set the IMU roll angle equal to the accelerometer roll angle when the quadcopter is started.
  angle_roll = angle_roll_acc;

  //Reset the PID controllers for a bumpless start.
  pid_i_mem_roll = 0;
  pid_last_roll_d_error = 0;
  pid_i_mem_pitch = 0;
  pid_last_pitch_d_error = 0;
  pid_i_mem_yaw = 0;
  pid_last_yaw_d_error = 0;
}

  // Stopping the motors: throttle low and yaw right.
if(start == 2 && receiver_input_channel_1 < 1050 && receiver_input_channel_4 > 1950)
  start = 0;
```

# 5 Etapa IV: Estabilização da Aeronave pelo Controle PID

O objetivo do algoritmo de controle é o de manter o quadricóptero estabilizado e nivelado enquanto não houverem comandos direcionais pelo piloto, além de inclinar e estabilizar a aeronave no sentido correto uma vez comandado pelo piloto. Uma visão geral da planta do quadricóptero com o sistema de controle pode ser visto na Figura 1.

## 5.1 Implementação do controlador PID

Para que o quadricóptero permaneça estabilizado e nivelado busca-se a condição em que as velocidades angulares medidas pelos giroscópios sejam zero, ou seja, iremos utilizar como erro do controlador ($\epsilon$) a diferença entre as velocidades angulares em cada eixo com a referência, ou *setpoint*, o qual para o quadricóptero permanecer nivelado é zero.

Aqui, tanto os valores das velocidades angulares quanto dos *setpoints*, devem estar em graus por

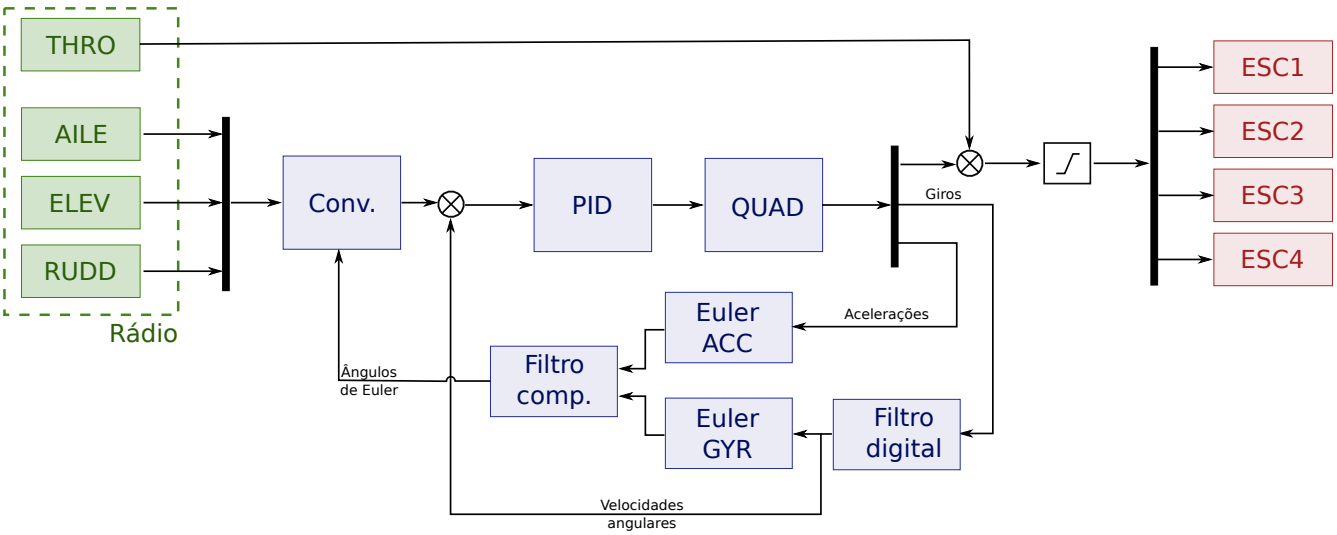



Figura 1: Visão geral da planta do quadricóptero.

segundo. Os erros referentes as velocidades angulares são expressas por:

$$\epsilon_\phi = p^{k+1} - p_{setpoint} \tag{14}$$

$$\epsilon_\theta = q^{k+1} - q_{setpoint} \tag{15}$$

$$\epsilon_\psi = r^{k+1} - r_{setpoint} \tag{16}$$

O valor dos *setpoints* diferente de zero em cada um dos eixos impõe um movimento de inclinação ao quadricóptero, ou seja, significa que mesmo com o quadricóptero em situação inicialmente nivelada, ao se forçar, por exemplo, em $p_{setpoint}$ um valor positivo resultará em erro $\epsilon_\phi$ negativo, logo o controlador interpretará como uma necessidade de inclinar o quadricóptero naquele eixo. Portanto, variando os valores dos *setpoints* por meio dos sinais de comando do rádio é possível ordenar a inclinação e giro do quadricóptero em cada um dos eixos.

Contudo, é importante que esse comando forçado de giro se auto-limite, ou seja, o movimento de inclinação se inicia com o quadricóptero nivelado e com um erro $\epsilon$ máximo devido ao comando do sinal do rádio no *setpoint*, e a medida que ele se inclina é importante que esse erro de inclinação diminua progressivamente até atingir uma inclinação máxima e permaneça inclinado, resultando em um movimento para frente, por exemplo.

Essas limitações de movimento são feitas por meio das equações a seguir:

$$p_{setpoint}(graus/s) = \frac{Receiver_{AILE}(\mu s) - 15\,\phi(graus)}{3} \tag{17}$$

$$q_{setpoint}(graus/s) = \frac{Receiver_{ELEV}(\mu s) - 15\,\theta(graus)}{3} \tag{18}$$

$$r_{setpoint}(graus/s) = \frac{Receiver_{RUDD}(\mu s) - 15\,\psi(graus)}{3} \tag{19}$$

Nestas equações os sinais de entrada de comandos de rádio *Receiver* ($\mu s$) devem ser ajustados para que quando o sinal de comando ficar abaixo do valor de centro de 1500 $\mu s$ resulte em um comando negativo, e se estiver acima do valor de centro de 1500 $\mu s$ resulte em um comando positivo. Além disso, é importante utilizar uma banda morta de 8 $\mu s$ positiva e negativa em torno do valor central de 1500 $\mu s$, para evitar comandos não intencionais. Esse ajuste é mostrado na equação seguinte:

$$Receiver = \begin{cases} Receiver_{Input} - 1508, & Receiver_{Input} > 1508 \\ Receiver_{Input} - 1492, & Receiver_{Input} < 1492 \end{cases} \tag{20}$$

Nos cálculos dos *setpoints* ainda se utilizou coeficientes de 15 multiplicando os ângulos de Euler em graus. Essa multiplicação representa uma conversão dos ângulos em um equivalente em $\mu s$. O valor de 15 equivale a uma inclinação máxima desejada de 32.8 graus quando houver diferença máxima em relação ao centro de 492 $\mu s$, ou seja, limite máximo de largura de pulso dos extremos em relação ao centro: $15 = (2000\mu s - 1508\mu s)/32.8 graus$. Portanto, valores acima de 15 implicam em uma inclinação máxima abaixo de 32.8 graus, e abaixo de 15 o contrário.

Há também a divisão da diferença por 3, esse coeficiente é uma conversão do resultado da diferença em cima em $\mu s$ para um valor em graus/s. Essa conversão é devido a um giro máximo de reação ao controle do sinal de rádio desejado de 164 graus/s, ou seja, $3 = (2000\mu s - 1508\mu s)/(164 graus/s)$. Portanto, coeficientes acima de 3 representam uma reação em giro máxima abaixo de 164 graus/s, enquanto abaixo de 3 é o contrário.

Ainda, o controlador utilizado será do tipo PID (Proporcional Integral Derivativo) para cada eixo de

movimento, cuja forma completa é:

$$PID_{\phi}\left(\mu s\right) = k_{P_{\phi}}\,\epsilon_{\phi}^{k} + k_{I_{\phi}}\left(\epsilon_{\phi}^{k} + \sum_{i=0}^{k-1}\epsilon_{\phi_i}\right) + k_{D_{\phi}}\left(\epsilon_{\phi}^{k} - \epsilon_{\phi}^{k-1}\right) \tag{21}$$

$$PID_{\theta}\left(\mu s\right) = k_{P_{\theta}}\,\epsilon_{\theta}^{k} + k_{I_{\theta}}\left(\epsilon_{\theta}^{k} + \sum_{i=0}^{k-1}\epsilon_{\theta_i}\right) + k_{D_{\theta}}\left(\epsilon_{\theta}^{k} - \epsilon_{\theta}^{k-1}\right) \tag{22}$$

$$PID_{\psi}\left(\mu s\right) = k_{P_{\psi}}\,\epsilon_{\psi}^{k} + k_{I_{\psi}}\left(\epsilon_{\psi}^{k} + \sum_{i=0}^{k-1}\epsilon_{\psi_i}\right) + k_{D_{\psi}}\left(\epsilon_{\psi}^{k} - \epsilon_{\psi}^{k-1}\right) \tag{23}$$

Nestas equações temos três categorias de ganhos para cada eixo de movimento, ou seja, ganhos proporcional ($k_P$), ganhos do integrador ($k_I$) e ganhos do derivativo ($k_D$), que multiplicam respectivamente o valor do erro do controlador, em seguida o valor da soma dos erros ao longo dos *loops* de controle, e por fim a diferença entre o erro atual e o erro do passo anterior.

Além disso, é importante estabelecer um limite superior e inferior à soma dos erros do integrador, cujo método é chamado saturação, ou seja, caso o somatório atinja um valor acima de certo limite superior, ou abaixo do limite inferior, força-se que o valor passe a ser igual ao limite correspondente. O limite utilizado é de 400 $\mu s$.

$$erro_I = \begin{cases} k_{I_{\phi}}\left(\epsilon_{\phi}^{k} + \sum_{i=0}^{k-1}\epsilon_{\phi_i}\right), & erro_{I_{min}} < erro_I < erro_{I_{max}} \\ erro_{I_{max}}, & erro_I \geq erro_{I_{max}} \\ erro_{I_{min}}, & erro_I \leq erro_{I_{min}} \end{cases} \tag{24}$$

Esses cálculos dos acionamentos devido aos controladores PID podem ser feitos por meio de uma função, como mostrado nas linhas de código a seguir.

```
void calculate_pid(){
  //Roll calculations
  pid_error_temp = gyro_roll_input - pid_roll_setpoint;
  pid_i_mem_roll += pid_i_gain_roll * pid_error_temp;
  if(pid_i_mem_roll > pid_max_roll)pid_i_mem_roll = pid_max_roll;
  else if(pid_i_mem_roll < pid_max_roll * -1)pid_i_mem_roll = pid_max_roll * -1;

  pid_output_roll = pid_p_gain_roll * pid_error_temp + pid_i_mem_roll +
  + pid_d_gain_roll * (pid_error_temp - pid_last_roll_d_error);
  if(pid_output_roll > pid_max_roll)pid_output_roll = pid_max_roll;
  else if(pid_output_roll < pid_max_roll * -1)pid_output_roll = pid_max_roll * -1;

  pid_last_roll_d_error = pid_error_temp;

  //Pitch calculations
  pid_error_temp = gyro_pitch_input - pid_pitch_setpoint;
  pid_i_mem_pitch += pid_i_gain_pitch * pid_error_temp;
  if(pid_i_mem_pitch > pid_max_pitch)pid_i_mem_pitch = pid_max_pitch;
  else if(pid_i_mem_pitch < pid_max_pitch * -1)pid_i_mem_pitch = pid_max_pitch * -1;

  pid_output_pitch = pid_p_gain_pitch * pid_error_temp + pid_i_mem_pitch +
  + pid_d_gain_pitch * (pid_error_temp - pid_last_pitch_d_error);
  if(pid_output_pitch > pid_max_pitch)pid_output_pitch = pid_max_pitch;
  else if(pid_output_pitch < pid_max_pitch * -1)pid_output_pitch = pid_max_pitch * -1;

  pid_last_pitch_d_error = pid_error_temp;

  //Yaw calculations
  pid_error_temp = gyro_yaw_input - pid_yaw_setpoint;
  pid_i_mem_yaw += pid_i_gain_yaw * pid_error_temp;
  if(pid_i_mem_yaw > pid_max_yaw)pid_i_mem_yaw = pid_max_yaw;
  else if(pid_i_mem_yaw < pid_max_yaw * -1)pid_i_mem_yaw = pid_max_yaw * -1;

  pid_output_yaw = pid_p_gain_yaw * pid_error_temp + pid_i_mem_yaw +
  + pid_d_gain_yaw * (pid_error_temp - pid_last_yaw_d_error);
  if(pid_output_yaw > pid_max_yaw)pid_output_yaw = pid_max_yaw;
  else if(pid_output_yaw < pid_max_yaw * -1)pid_output_yaw = pid_max_yaw * -1;

  pid_last_yaw_d_error = pid_error_temp;
}
```

Por fim, os valores devem ser aplicados para cada um dos ESCs dos motores, como abaixo. Aqui, a largura de pulso Throttle ($\mu s$) deve vir direto do controle do rádio normalizado.

Ainda, é importante deixar uma margem aos incrementos devido ao controlador PID em relação ao sinal do Throttle ($\mu s$), visto que o sinal máximo a cada ESC é 2000 $\mu s$. Assim, limita-se o sinal máximo permitido ao Throttle ($\mu s$) de 1800 $\mu s$.

Logo,

$$Se\, Throttle(\mu s) > 1800 => Throttle(\mu s) = 1800. \tag{25}$$

$$esc_1\,(\mu s) = Throttle\,(\mu s) + PID_\phi\,(\mu s) - PID_\theta\,(\mu s) - PID_\psi\,(\mu s) \tag{26}$$

$$esc_2\,(\mu s) = Throttle\,(\mu s) + PID_\phi\,(\mu s) + PID_\theta\,(\mu s) + PID_\psi\,(\mu s) \tag{27}$$

$$esc_3\,(\mu s) = Throttle\,(\mu s) - PID_\phi\,(\mu s) + PID_\theta\,(\mu s) - PID_\psi\,(\mu s) \tag{28}$$

$$esc_4\,(\mu s) = Throttle\,(\mu s) - PID_\phi\,(\mu s) - PID_\theta\,(\mu s) + PID_\psi\,(\mu s) \tag{29}$$

Também é importante fazer a saturação do sinal aos ESCs resultante do PID, assim, caso o resultado dê abaixo de 1100 $\mu s$, deve-se forçar 1100 $\mu s$ para que os motores não desliguem em voo, e também, caso o resultado dê acima de 2000 $\mu s$, deve-se forçar 2000 $\mu s$, valor máximo permitido ao ESC.

Portanto, pode-se observar nas linhas de código seguintes a implementação do algoritmo. Note que o sinal aos ESCs pode assumir valores diferentes de 1000 $\mu s$ apenas se a variável $start = 2$.

```
  if (start == 2){                                                    //The motors are started.
    if (throttle > 1800) throttle = 1800;
    esc_1 = throttle + pid_output_roll - pid_output_pitch - pid_output_yaw;
    esc_2 = throttle + pid_output_roll + pid_output_pitch + pid_output_yaw;
    esc_3 = throttle - pid_output_roll + pid_output_pitch - pid_output_yaw;
    esc_4 = throttle - pid_output_roll - pid_output_pitch + pid_output_yaw;

    if (battery_voltage < 1240 && battery_voltage > 800){             //Is the battery connected?
      esc_1 += esc_1 * ((1240 - battery_voltage)/(float)3500);
      esc_2 += esc_2 * ((1240 - battery_voltage)/(float)3500);
      esc_3 += esc_3 * ((1240 - battery_voltage)/(float)3500);
      esc_4 += esc_4 * ((1240 - battery_voltage)/(float)3500);
    }

    if (esc_1 < 1100) esc_1 = 1100;   //Keep the motors running.
    if (esc_2 < 1100) esc_2 = 1100;   //Keep the motors running.
    if (esc_3 < 1100) esc_3 = 1100;   //Keep the motors running.
    if (esc_4 < 1100) esc_4 = 1100;   //Keep the motors running.

    if(esc_1 > 2000)esc_1 = 2000;    //Limit the esc-1 pulse to 2000us.
    if(esc_2 > 2000)esc_2 = 2000;    //Limit the esc-2 pulse to 2000us.
    if(esc_3 > 2000)esc_3 = 2000;    //Limit the esc-3 pulse to 2000us.
    if(esc_4 > 2000)esc_4 = 2000;    //Limit the esc-4 pulse to 2000us.
  }

  else{
    esc_1 = 1000;   //If start is not 2 keep a 1000us pulse for ess-1.
    esc_2 = 1000;   //If start is not 2 keep a 1000us pulse for ess-2.
    esc_3 = 1000;   //If start is not 2 keep a 1000us pulse for ess-3.
    esc_4 = 1000;   //If start is not 2 keep a 1000us pulse for ess-4.
  }
```

## 5.2 Leitura da voltagem da bateria

A bateria 3S quando totalmente carregada apresenta 12.6 V, o que corresponde a uma entrada de 5V na porta A0 do Arduino. As portas analógicas do Arduino Uno convertem 0-5V para valores de 0-1023. Dessa forma, a voltagem da bateria pode ser calculada como:

$$V_{batt} = (A_0 + V_{diode}) \frac{1260}{1023} \tag{30}$$

sendo $A_0$ o valor do pino A0 e $V_{diode}$ a compensação pela queda de tensão no diodo. O valor obtido a partir de (30) representa a voltagem multiplicada por um fator 100, ou seja, o valor de 1030 corresponde a uma voltagem $V_{batt} = 10.3\ V$ da bateria.

Ainda, é importante aplicar também um filtro digital na leitura da tensão da bateria para se remover ruídos indesejados.

$$V_{batt}^{k+1} = 0.92\, V_{batt}^{k} + 0.08\, V_{batt}^{new} \tag{31}$$

A medida da tensão da bateria é feita para se avisar o piloto por meio de um LED de que a tensão está baixa, no caso abaixo de 10 V. Portanto, quando a variável $V_{batt} < 1000$, deve-se enviar um sinal de nível lógico alto para a saída digital do LED.

Por último, a medida que a tensão da bateria se reduz, é importante atualizar o sinal de comando aos esc's para que a tração permaneça a mesma, isso é feito utilizando as equações seguintes, para tensão medida abaixo de 12.4 V.

$$esc_1 = esc_1 + esc_1\,(1240 - V_{batt})/3500 \tag{32}$$

$$esc_2 = esc_2 + esc_2\,(1240 - V_{batt})/3500 \tag{33}$$

$$esc_3 = esc_3 + esc_3\,(1240 - V_{batt})/3500 \tag{34}$$

$$esc_4 = esc_4 + esc_4\,(1240 - V_{batt})/3500 \tag{35}$$

### 5.3 Geração dos Pulsos aos ESCs

Uma vez calculadas as larguras dos pulsos a serem aplicadas a cada ESC, deve-se gerar esses pulsos por meio das saídas das portas digitais correspondentes. Isso é feito, seguindo o procedimento:

- Partindo inicialmente de todas as saídas digitais aos ESCs correspondentes no nível lógico 0 (LOW).
- Faz-se a leitura do tempo imediatamente antes de se iniciar os pulsos gravando o valor na variável *loop_timer*.
- Define-se todas as saídas digitais aos ESCs no nível lógico 1 (HIGH) simultaneamente.
- Calcula-se o tempo final de cada pulso sendo a soma do tempo medido anteriormente (*loop_timer*) com a largura do pulso calculada pelo controlador a cada ESC.
- Continuamente faz-se a leitura do tempo e quando ele ficar maior que o tempo final a cada pulso, desliga-se a porta digital correspondente (nível lógico 0 - LOW).
- Deve-se aguardar até que todos os quatro pulsos retornem ao nível lógico 0 para continuar com os passos seguintes do código.

O *loop* de controle foi desenvolvido para levar 4000 $\mu s$ em cada ciclo, ou seja, esse é o tempo máximo para se fazer todos os procedimentos do código de controle a serem repetidos continuamente. Isso significa

que a frequência do *loop* de controle é 250 Hz. Portanto, caso algum procedimento tenha alguma falha, o tempo do *loop* pode ultrapassar os 4000 $\mu s$, o que pode afetar o controle da aeronave. Deste modo, uma forma de indicar que algum erro pode ter ocorrido é o de determinar o acendimento do LED caso esse *loop* ultrapasse esse período.

As linhas de código seguintes resumem os procedimentos descritos.

```
1  if(micros() - loop_timer > 4050)digitalWrite(12, HIGH);  //Turn on LED if loop time exceeds 4050us.
2
3  //All the information for controlling the motor's is available.
4  //The refresh rate is 250Hz. That means the esc's need there pulse every 4ms.
5  while(micros() - loop_timer < 4000);   //We wait until 4000us are passed.
6  loop_timer = micros();                 //Set the timer for the next loop.
7
8  PORTD |= B11110000;                    //Set digital outputs 4,5,6 and 7 high.
9  timer_channel_1 = esc_1 + loop_timer;  //Calculate the time of the faling edge of the esc-1 pulse.
10 timer_channel_2 = esc_2 + loop_timer;  //Calculate the time of the faling edge of the esc-2 pulse.
11 timer_channel_3 = esc_3 + loop_timer;  //Calculate the time of the faling edge of the esc-3 pulse.
12 timer_channel_4 = esc_4 + loop_timer;  //Calculate the time of the faling edge of the esc-4 pulse.
13
14 while(PORTD >= 16){                        //Stay in this loop until output 4,5,6 and 7 are low.
15   esc_loop_timer = micros();                                  //Read the current time.
16   if(timer_channel_1 <= esc_loop_timer)PORTD &= B11101111;  //Digital output 4 low if time is expired.
17   if(timer_channel_2 <= esc_loop_timer)PORTD &= B11011111;  //Digital output 5 low if time is expired.
18   if(timer_channel_3 <= esc_loop_timer)PORTD &= B10111111;  //Digital output 6 low if time is expired.
19   if(timer_channel_4 <= esc_loop_timer)PORTD &= B01111111;  //Digital output 7 low if time is expired.
20 }
```

## 5.4 Visão geral do código

**Procedimentos de inicialização do drone**

1. Leitura dos dados da EEPROM (limites do rádio e offsets dos giroscópios)
2. Inicializar a comunicação com a MPU6050
3. Definir as portas 4, 5, 6 e 7 como saídas (controle dos ESCs)
4. Definir a porta 12 como saída (LED)
5. Acende o LED para iniciar o início da configuração
6. Configuração da MPU6050
7. Inicialização das portas de controle dos ESCs
8. Determinação dos offsets do giroscópio (média de 2000 - 3000 leituras)
9. Configuração das interrupções
10. Leitura da voltagem da bateria

11. Pisca o LED por 5 segundos para indicar que a configuração inicial foi finalizada
12. Fim da configuração

**Laço principal do programa**

1. Ler as informações da IMU
2. Composição das velocidades angulares
3. Determinação do ângulo a partir das velocidades angulares (integração temporal)
4. Cálculo da aceleração resultante
5. Cálculo dos ângulos a partir da leitura do acelerômetro
6. Correção de *drift* dos ângulos
7. Cálculo Final dos Ângulos (Filtro Complementar)
8. Checagem da posição dos controles (0 e 1: turn off, 2: turn on)
9. Verifica as entradas e estabelece uma zona morta de $\pm 8\mu s$
10. Calcula os parâmetros do controlador PID
11. Verifica o nível da bateria
12. Ajusta o motor com o valor do canal THRO
13. Ajusta os ESCs com os valores dos ganhos dos controladores
14. Acender o LED se o valor do *loop* for excedido
15. Calcular a largura de pulso para cada ESC
16. Enviar os valores para os ESCs

# A Códigos auxiliares

## A.1 Configuração da IMU

```
void set_gyro_registers(){
  //Setup the MPU-6050
  if(eeprom_data[31] == 1){
    Wire.beginTransmission(gyro_address);   //Start communication with the address found during search.
    Wire.write(0x6B);   //We want to write to the PWR_MGMT_1 register (6B hex)
    Wire.write(0x00);   //Set the register bits as 00000000 to activate the gyro
    Wire.endTransmission();     //End the transmission with the gyro.

    Wire.beginTransmission(gyro_address);    //Start communication with the address found during search.
    Wire.write(0x1B);   //We want to write to the GYRO_CONFIG register (1B hex)
    Wire.write(0x08);   //Set the register bits as 00001000 (500dps full scale)
    Wire.endTransmission();     //End the transmission with the gyro

    Wire.beginTransmission(gyro_address);    //Start communication with the address found during search.
    Wire.write(0x1C);   //We want to write to the ACCEL_CONFIG register (1A hex)
    Wire.write(0x10);   //Set the register bits as 00010000 (+/- 8g full scale range)
    Wire.endTransmission();     //End the transmission with the gyro

    //Let's perform a random register check to see if the values are written correct
    Wire.beginTransmission(gyro_address);  //Start communication with the address found during search
    Wire.write(0x1B);   //Start reading @ register 0x1B
    Wire.endTransmission();     //End the transmission
    Wire.requestFrom(gyro_address, 1);   //Request 1 bytes from the gyro
    while(Wire.available() < 1);//Wait until the 6 bytes are received
    if(Wire.read() != 0x08){    //Check if the value is 0x08
      digitalWrite(12,HIGH);    //Turn on the warning led
      while(1)delay(10);//Stay in this loop for ever
    }

    Wire.beginTransmission(gyro_address);    //Start communication with the address found during search
    Wire.write(0x1A);   //We want to write to the CONFIG register (1A hex)
    Wire.write(0x03);   //Set the register bits as 00000011 (Set Digital Low Pass Filter to ~43Hz)
    Wire.endTransmission();     //End the transmission with the gyro

  }
}
```

## A.2 Leitura dos valores da IMU

```
void gyro_signalen(){
  //Read the MPU-6050
  Wire.beginTransmission(gyro_address);     //Start communication with the gyro.
  Wire.write(0x3B);          //Start reading @ register 43h and auto increment with every read.
  Wire.endTransmission();           //End the transmission.
  Wire.requestFrom(gyro_address,14); //Request 14 bytes from the gyro.

  while(Wire.available() < 14);             //Wait until the 14 bytes are received.
  acc_axis[1] = Wire.read()<<8|Wire.read(); //Add the low and high byte to the acc_x variable.
  acc_axis[2] = Wire.read()<<8|Wire.read(); //Add the low and high byte to the acc_y variable.
  acc_axis[3] = Wire.read()<<8|Wire.read(); //Add the low and high byte to the acc_z variable.
  temperature = Wire.read()<<8|Wire.read(); //Add the low and high byte to the temperature variable.
  gyro_axis[1] = Wire.read()<<8|Wire.read();//Read high and low part of the angular data.
  gyro_axis[2] = Wire.read()<<8|Wire.read();//Read high and low part of the angular data.
  gyro_axis[3] = Wire.read()<<8|Wire.read();//Read high and low part of the angular data.

  if(cal_int == 2000){
    gyro_axis[1] -= gyro_axis_cal[1];  //Only compensate after the calibration.
    gyro_axis[2] -= gyro_axis_cal[2];  //Only compensate after the calibration.
    gyro_axis[3] -= gyro_axis_cal[3];   //Only compensate after the calibration.
  }

}
```

## A.3 Interrupções para leitura dos códigos

```
ISR(PCINT0_vect){
  current_time = micros();
  //Channel 1=========================================
  if(PINB & B00000001){             //Is input 8 high?
    if(last_channel_1 == 0){        //Input 8 changed from 0 to 1.
      last_channel_1 = 1;           //Remember current input state.
      timer_1 = current_time;       //Set timer_1 to current_time.
    }
  }
  else if(last_channel_1 == 1){     //Input 8 is not high and changed from 1 to 0.
    last_channel_1 = 0;             //Remember current input state.
    receiver_input[1] = current_time - timer_1;     //Channel 1 is current_time - timer_1.
  }
  //Channel 2=========================================
  if(PINB & B00000010 ){            //Is input 9 high?
    if(last_channel_2 == 0){        //Input 9 changed from 0 to 1.
```

```
      last_channel_2 = 1;           //Remember current input state.
      timer_2 = current_time;       //Set timer_2 to current_time.
    }
  }
  else if(last_channel_2 == 1){     //Input 9 is not high and changed from 1 to 0.
    last_channel_2 = 0;             //Remember current input state.
    receiver_input[2] = current_time - timer_2;     //Channel 2 is current_time - timer_2.
  }
  //Channel 3=========================================
  if(PINB & B00000100 ){            //Is input 10 high?
    if(last_channel_3 == 0){        //Input 10 changed from 0 to 1.
      last_channel_3 = 1;           //Remember current input state.
      timer_3 = current_time;       //Set timer_3 to current_time.
    }
  }
  else if(last_channel_3 == 1){     //Input 10 is not high and changed from 1 to 0.
    last_channel_3 = 0;             //Remember current input state.
    receiver_input[3] = current_time - timer_3;     //Channel 3 is current_time - timer_3.

  }
  //Channel 4=========================================
  if(PINB & B00001000 ){            //Is input 11 high?
    if(last_channel_4 == 0){        //Input 11 changed from 0 to 1.
      last_channel_4 = 1;           //Remember current input state.
      timer_4 = current_time;       //Set timer_4 to current_time.
    }
  }
  else if(last_channel_4 == 1){     //Input 11 is not high and changed from 1 to 0.
    last_channel_4 = 0;             //Remember current input state.
    receiver_input[4] = current_time - timer_4;     //Channel 4 is current_time - timer_4.
  }
}
```

# B Variáveis globais

Tabela 2: Variáveis globais

| Nome | Tipo | Descrição |
|---|---|---|
| `last_channel_1` | byte | Controle de interrupções |
| `last_channel_2` | | |
| `last_channel_3` | | |
| `last_channel_4` | | |
| `receiver_input_channel_1` | volatile int | Largura de pulso dos sinais de rádio |
| `receiver_input_channel_2` | | |
| `receiver_input_channel_3` | | |
| `receiver_input_channel_4` | | |
| `center_channel_1` | int | Largura de pulso centro |
| `center_channel_2` | | |
| `center_channel_3` | | |
| `center_channel_4` | | |
| `high_channel_1` | int | Máxima largura de pulso do sinal de rádio |
| `high_channel_2` | | |
| `high_channel_3` | | |
| `high_channel_4` | | |
| `low_channel_1` | int | Mínima largura de pulso do sinal de rádio |
| `low_channel_2` | | |
| `low_channel_3` | | |
| `low_channel_4` | | |
| `esc_1` | int | Larguras de pulso enviadas para os ESCs |
| `esc_2` | | |
| `esc_3` | | |
| `esc_4` | | |
| `throttle` | int | Largura de pulso referente a entrada do motor |
| `battery_voltage` | int | Voltagem da bateria |
| `timer_channel_1` | unsigned long | Controlador de tempo dos sinais dos ESCs |
| `timer_channel_2` | | |
| `timer_channel_3` | | |
| `timer_channel_4` | | |

Tabela 3: Variáveis globais - Cont.

| Nome | Tipo | Descrição |
|---|---|---|
| `acc_x`<br>`acc_y`<br>`acc_z` | long | Acelerações oriundas da IMU |
| `loop_timer` | unsigned long | Variável para controlar o tempo do loop principal |
| `gyro_pitch`<br>`gyro_roll`<br>`gyro_yaw` | double | Velocidades angulares |
| `gyro_pitch_offset`<br>`gyro_roll_offset`<br>`gyro_yaw_offset` | double | Offsets referentes às velocidades angulares |
| `gyro_pitch_input`<br>`gyro_roll_input`<br>`gyro_yaw_input` | double | Velocidades angulares filtradas |
| `pid_output_roll`<br>`pid_output_pitch`<br>`pid_output_yaw` | float | Resultado dos controladores PID |
| `pid_last_roll_d_error`<br>`pid_last_pitch_d_error`<br>`pid_last_yaw_d_error` | float | Último erro do controlador derivativo |
| `pid_roll_setpoint`<br>`pid_pitch_setpoint`<br>`pid_yaw_setpoint` | float | *Setpoints* de referência para cálculo dos erros do PID |
| `pid_i_mem_roll`<br>`pid_i_mem_pitch`<br>`pid_i_mem_yaw` | float | Somatório dos erros do controlador integrador |
| `angle_pitch`<br>`angle_roll` | float | Ângulos finais de atitude |
| `angle_pitch_acc`<br>`angle_roll_acc` | float | Ângulos de atitude devido aos acelerômetros |